



Appunti di Basi di Dati

Nicholas Scorrano

Anno Accademico 2025/2026

Indice

1	Algebra Relazionale	7
1.1	Cosa è?	7
1.2	Struttura base di una relazione	7
1.3	Chiavi e Vincoli	7
1.4	Categorie di operatori	8
1.4.1	Precedenza e parentesizzazione	8
1.5	Unione $\cup$	9
1.5.1	Definizione	9
1.5.2	Esempio	9
1.6	Intersezione $\cap$	9
1.6.1	Definizione	9
1.6.2	Esempio	9
1.7	Differenze	9
1.7.1	Definizione	9
1.7.2	Esempio	10
1.8	Ridenominazione ρ	10
1.8.1	Definizione	10
1.8.2	Esempio	10
1.9	Selezione σ	10
1.9.1	Definizione	10
1.9.2	Semantica	11
1.9.3	Esempio	11
1.10	Proiezione π	11
1.10.1	Definizione	11
1.10.2	Esempio	11
1.11	Prodotto cartesiano $\times$	12
1.11.1	Definizione	12
1.11.2	Esempio	12
1.12	Theta-join $\bowtie_{\theta}$	12
1.12.1	Definizione	12
1.12.2	Esempio	12
1.13	Natural-join $\bowtie$	13
1.13.1	Definizione	13
1.13.2	Esempio	13
1.14	Alberi di esecuzione	14

1.14.1	Definizione	14
1.14.2	Esempio	14
1.15	Divisione $\div$	14
1.15.1	Definizione	14
1.15.2	Esempio	14
1.16	Aggregazione (Y)	15
1.16.1	Definizione	15
1.16.2	Esempio	15

2 DDL - Data Definition Language 16

2.1	Notazione	16
2.2	Schemi	16
2.2.1	Cosa sono?	16
2.2.2	Sintassi di creazione	16
2.2.3	Esempio: Creazione di uno schema vuoto	16
2.2.4	Esempio: Creazione di uno schema utilizzando tutte le opzioni disponibili	16
2.3	Tabelle	17
2.3.1	Cosa sono?	17
2.3.2	Sintassi di creazione	17
2.3.3	Esempio: Creazione di una tabella con elementi minimi	17
2.3.4	Esempio: Creazione di una tabella full optional	17
2.4	Definizione dei Dati: I Domini	18
2.4.1	Cosa sono?	18
2.4.2	Domini elementari	18
2.5	Domini definiti dagli utenti	18
2.5.1	Cosa sono?	18
2.5.2	Sintassi di creazione	18
2.5.3	Esempio: Creazione di dominio semplice	18
2.5.4	Esempio: Creazione di un dominio full optional	19
2.6	Vincoli intrarelazionale	19
2.6.1	Tipidi di vincoli intrarelazionali	19
2.6.2	Esempio	19
2.7	Vincoli interrelazionali: Integrità referenziale	20
2.7.1	Cosa sono?	20
2.7.2	Sintassi	20
2.7.3	Esempio	20
2.8	Aggiornamento	21
2.8.1	Il problema dell'aggiornamento	21
2.8.2	Il problema delle violazioni	21
2.8.3	Reazioni a violazioni sulla tabelal padre	21

2.8.4	Vincoli con nomi	22
2.9	Modifiche degli schemi: ALTER	22
2.9.1	A cosa serve?	22
2.9.2	Tipi di ALTER	22
2.9.3	Sintassi ALTER DOMAIN	22
2.9.4	Esempio di ALTER DOMAIN	22
2.9.5	Sintassi ALTER TABLE	23
2.9.6	Esempio di ALTER TABLE	23
2.10	Vincolo interrelazionale: UPDATE	23
2.10.1	A cosa serve	23
2.10.2	Esempi pratici	24
2.11	Vincolo interrelazionale: DROP	25
2.11.1	Cosa fa?	25
2.11.2	DROP DOMAIN	25
3	Operatore SELECT - Interrogazione di un database	27
3.1	A cosa serve?	27
3.2	Sintassi	27
3.3	Esempio	27
3.4	Istruzione AS	27
3.4.1	A cosa serve?	27
3.4.2	Sintassi	27
3.4.3	Esempio	27
3.5	Clausola FROM	28
3.5.1	Sintassi	28
3.5.2	Notazione punto	28
3.5.3	Esempio	28
3.6	Clausola WHERE	28
3.6.1	A cosa serve?	28
3.6.2	Operatori di confronto	28
3.6.3	Operatore LIKE	28
3.6.4	Operatore BETWEEN	29
3.6.5	Operatore IN	29
3.6.6	Predicati semplici	29
3.6.7	Precedenza di valutazione tra operatori booleani	29
3.6.8	Sintassi	29
3.6.9	Esempio generico	29
3.6.10	Esempio con LIKE	29
3.6.11	Esempio con BETWEEN	30
3.6.12	Esempio con IN	30

3.7	Clausola DISTINCT	30
3.7.1	A cosa serve?	30
3.7.2	Sintassi	30
3.7.3	Esempio	30
3.8	Espressioni	30
3.8.1	A cosa servono?	30
3.8.2	Funzioni predefinite	31
3.8.3	Funzioni per stringhe	31
3.8.4	Esempio di utilizzo di funzioni aritmetiche	31
3.8.5	Esempio di utilizzo di funzioni per stringhe	31
4	Operatore JOIN	32
4.1	A cosa serve?	32
4.1.1	Cosa è un predicato join?	32
4.2	Sintassi JOIN Legacy	32
4.2.1	Esempio	32
4.3	Sintassi JOIN SQL-2	32
4.3.1	Esempio	33
4.4	OUTER JOIN	33
4.4.1	Cosa fa?	33
4.4.2	LEFT OUTER JOIN (o LEFT JOIN)	33
4.4.3	RIGHT JOIN	33
4.5	Self Join	34
4.5.1	Cosa è?	34
4.5.2	A cosa serve?	34
4.5.3	Sintassi	35
4.5.4	Esempio	35
4.6	Join Esplicito	35
4.6.1	Cosa è?	35
4.6.2	A cosa serve?	35
4.6.3	Sintassi	35
4.6.4	Esempio	35
5	Ordinamento dei risultati	36
5.1	ORDER BY	36
5.1.1	Cosa è?	36
5.1.2	Sintassi	36
5.1.3	Esempio	36
5.2	Operatori di aggregazione	36
5.2.1	Quali sono?	36

5.2.2	Come vengono applicati?	37
5.3	COUNT	37
5.3.1	Cosa fa?	37
5.3.2	Sintassi	37
5.3.3	Esempio	37
5.4	SUM	37
5.4.1	Cosa fa?	37
5.4.2	Sintassi	38
5.4.3	Esempio	38
5.5	MIN/MAX	38
5.5.1	Cosa fa?	38
5.5.2	Sintassi	38
5.5.3	Esempio	38
5.6	AVG	38
5.6.1	Cosa fa?	38
5.6.2	Sintassi	38
5.6.3	Esempio	39
5.7	Target List	39
5.8	Operatore di aggregamento GROUP BY	39
5.8.1	Cosa è?	39
5.8.2	Sintassi	39
5.8.3	Esempio	39
5.9	Condizioni WHERE e HAVING	39
5.9.1	Sintassi WHERE	40
5.9.2	Esempio di uso di WHERE	40
5.9.3	Sintassi HAVING	40
5.9.4	Esempio di uso di HAVING	40
6	Interrogazioni di tipi insiemistico	41
6.1	Operatori insiemistici	41
6.1.1	Sintassi	41
6.2	UNION	41
6.2.1	Cosa fa?	41
6.2.2	Sintassi	41
6.2.3	Esempio	41
6.3	INTERSECT	42
6.3.1	Cosa fa?	42
6.3.2	Sintassi	42
6.3.3	Esempio	42
6.4	EXCEPT	42

6.4.1	Cosa fa?	42
6.4.2	Sintassi	43
6.4.3	Esempio	43
7	Interrogazioni Nidificate	44
7.1	Cosa sono?	44
7.2	Come funzionano?	44
7.3	Sintassi Subquery	44
7.4	Predicati ANY e ALL	44
7.4.1	Sintassi	44
7.4.2	Esempio	45
7.5	Quantificatori esistenziali	45
7.5.1	EXISTS	45
7.5.2	NOT EXISTS	45
8	Viste	47
8.1	Cosa sono?	47
8.2	Proprietà fondamentali	47
8.3	A cosa servono?	47
8.4	Sintassi	47
8.5	Esempio creazione vista	48

1. Algebra Relazionale

1.1 Cosa è?

L'algebra relazionale è un linguaggio procedurale basato su concetti di tipo algebrico, le operazioni sono definite su relazioni e producono a loro volta altre relazioni.

Si compone di vari operatori fondamentali, che permettono di comporre operazioni in espressioni e alberi di esecuzione per integrazioni anche molto complesse

1.2 Struttura base di una relazione

Una relazione è un'astrazione matematica che definisce un insieme di tuple, ciascuna definita su uno schema comune

- **Schema di relazioni** $R(X)$
 - R = Nome della relazione
 - X = Insiemi di attributi $(A_1, A_2, \dots, A_n)$ ciascuno associato a un dominio di valori
- **Tupla:** Istanza di R , ossia un vettore ordinato di valori $(v_1, v_2, \dots, v_n)$ in $R(X)$
- **Dominio:** Insieme di valori leciti per ciascun attributo
- **Cardinalità** $|R|$: Numero di tuple in R
- **Grado** $|X|$: Numero di attributi di R
- **Tabella:** Rappresentazione "intuitiva" di R , dove le righe sono le tuple e le colonne sono gli attributi

1.3 Chiavi e Vincoli

- **Superchiave:** Insieme di attributi $K \subseteq X$ tale che non esistono 2 tuple distinte t_1, t_2 con $t_1[K] = t_2[K]$
- **Chiave:** Superchiave minimale (nessun sottoinsieme proprio di K è superchiave)
- **Chiavi candidate:** Tutte le chiavi possibili, una di queste diventa primaria
- **Vincoli di integrità referenziale:**
 - Si dichiara $\text{FOREIGN KEY}(A_1, \dots, A_n) \text{ REFERENCES}(B_1, \dots, B_n)$
 - Impone che ogni valore non-NULL di $((A_1, \dots, A_n))$ in R appaia in S su $(B_1, \dots, B_n)$
- **Vincoli di tupla:** Espressioni booleane su attributi
- **Vincoli di dominio:** es $0 \leq \text{Voto} \leq 30$

1.4 Categorie di operatori

- **Operatori Insiemistici**(richiedono schemi identici)
 - $\cap$ Unione
 - $\cup$ Intersezione
 - $-$ Differenza
- **Operatori unari**(agiscono su una sola relazione)
 - σ Selezione: Filtra le tuple che soddisfano una condizione (taglio orizzontale)
 - π Proiezione: Estrae un sottoinsieme di attributi, eliminando duplicati (taglio verticale)
 - ρ Ridenominazione: Rinomina relazione e/o attributi per armonizzare schemi
- **Operatori Binari**(combina due relazioni)
 - $\times$ Prodotto cartesiano: Accoppia ogni tupla di R con ogni tupla di S
 - $\bowtie_{\theta}$ Theta-Join: Join con condizione generica
 - $\bowtie$ Natural-Join: Join automatico su attributo omonimi
- **Operatori Derivati**(costruiti via quelli fondamentali)
 - $\div$ Divisione: Usata per query universali

1.4.1 Precedenza e parentesizzazione

Convenzioni di precedenza (senza parentesi)

1. σ, π, ρ (unari)
2. $\bowtie, \times$ (binari "forti")
3. $\cap, \cup$ (insiemistici)

1.5 Unione $\cup$

1.5.1 Definizione

$$R \cup S = \{t | t \in R \text{ AND } t \in S\}$$

- È un'operazione insiemistica: elimina i duplicati
- Richiede schemi identici (stessi attributi e domini)
- In SQL corrisponde a UNION

1.5.2 Esempio

$$R : (\text{ID}, \text{Nome}) = \{(1, \text{Alice}, (2, \text{Bob}))\} \quad (1.1)$$

$$S : (\text{ID}, \text{Nome}) = \{(2, \text{Bob}, (3, \text{Claudia}))\} \quad (1.2)$$

$$R \cup S = \{(1, \text{Alice}, (2, \text{Bob})), (3, \text{Claudia})\} \quad (1.3)$$

1.6 Intersezione $\cap$

1.6.1 Definizione

$$R \cap S = \{t | t \in R \text{ OR } t \in S\}$$

- Trova tutte le tuple comuni
- Richiede ancora schemi identici
- In SQL corrisponde a INTERSECT

1.6.2 Esempio

$$R : (\text{ID}, \text{Nome}) = \{(1, \text{Alice}, (2, \text{Bob}))\} \quad (1.4)$$

$$S : (\text{ID}, \text{Nome}) = \{(2, \text{Bob}), (3, \text{Claudia})\} \quad (1.5)$$

$$R \cap S = \{(2, \text{Bob})\} \quad (1.6)$$

1.7 Differenze

1.7.1 Definizione

$$R - S = \{t | t \in R \text{ AND } t \notin S\}$$

- Non commutativa: $R - S \neq S - R$
- Consente di rimuovere tuple indesiderate
- In SQL corrisponde a EXCEPT (o MINUS)

1.7.2 Esempio

$$R : (\text{ID}, \text{Nome}) = \{(1, \text{Alice}), (2, \text{Bob})\} \quad (1.7)$$

$$S : (\text{ID}, \text{Nome}) = \{(2, \text{Bob}), (3, \text{Claudia})\} \quad (1.8)$$

$$R - S = \{(2, \text{Bob})\} \quad (1.9)$$

1.8 Ridenominazione ρ

1.8.1 Definizione

- $\rho_{B_1, \dots, B_n \leftarrow A_1, \dots, A_n}(R)$ cambia solo i nomi degli attributi $A_i \rightarrow B_i$, lasciando inalterati i valori
- Utili per:
 - Preparare operazioni insiemistiche (schemi compatibili)
 - Disambiguare auto-join
- Se volessimo cambiare anche il nome della relazioni in R' , allora $R' \leftarrow \rho(R)$ oppure $\rho(R) \rightarrow R'$

1.8.2 Esempio

Padre	Figlio
Adamo	Caino
Adamo	Abele
Abramo	Isacco
Abramo	Ismaele

PATERNITÀ

Madre	Figlio
Eva	Caino
Eva	Set
Sara	Isacco
Agar	Ismaele

MATERNITÀ

Genitore	Figlio
Adamo	Caino
Adamo	Abele
Abramo	Isacco
Abramo	Ismaele
Eva	Caino
Eva	Set
Sara	Isacco
Agar	Ismaele

$$\rho_{\text{Genitore} \leftarrow \text{Padre}}(\text{PATERNITÀ}) \cup \rho_{\text{Genitore} \leftarrow \text{Madre}}(\text{MATERNITÀ})$$

1.9 Selezione σ

1.9.1 Definizione

$$\sigma_F(R)$$

- Filtra tuple in base ad una condizione F
- Predicati:
 - $A \theta B$ oppure $A \theta c$, con $\theta \in \{=, \neq, >, <, \geq, \leq\}$ è un operatore di confronto, A e B sono attributi di su cui il confronto θ abbia un senso, e c'è una costante compatibile con il dominio di A
 - combinati $\wedge$ (AND), $\vee$ (OR), $\neg$ (NOT)

1.9.2 Semantica

- Mantiene stesso schema di R
- Risultato: sottoinsieme delle tuple(o righe) di R
- Riduce la cardinalità, dunque abbiamo un'ottimizzazione se la spingiamo in anticipo (cioè, abbiamo meno tuple su cui applicare la sotto-operazione)

1.9.3 Esempio

ID	Età	Reparto
1	28	IT
2	35	HR
3	42	IT

R

ID	Età	Reparto
2	35	HR
3	42	IT

$\sigma_{\text{Età} > 30}(R)$

ID	Età	Reparto
3	42	IT

$\sigma_{\text{Età} > 30 \wedge \text{Reparto} = 'IT'}(R)$

1.10 Proiezione π

1.10.1 Definizione

$\pi_{A_1, \dots, A_n}(R)$

- Seleziona un sottoinsieme di attributi (colonne)
- Elimina i duplicati(semantica insiemistica)

1.10.2 Esempio

ID	Età	Reparto
1	28	IT
2	35	HR
3	42	IT

R

Reparto
IT
HR

$\pi_{\text{Reparto}}(R)$

1.11 Prodotto cartesiano $\times$

1.11.1 Definizione

$$R \times S = \{(t, u) | t \in R \wedge u \in S\} \quad (\text{concatena tuple})$$

- Schema: attributi di R unito agli attributi di S
- Cardinalità: $|R| \cdot |S|$

1.11.2 Esempio

ID	Cliente
1	Andrea
2	Barbara

R

Codice	Prodotto	Quantità
A	Scarpe	100
B	Magliette	200
C	Occhiali	300

S

ID	Cliente	Codice	Prodotto	Quantità
1	Andrea	A	Scarpe	100
1	Andrea	B	Magliette	200
1	Andrea	C	Occhiali	300
2	Barbara	A	Scarpe	100
2	Barbara	B	Magliette	200
2	Barbara	C	Occhiali	300

$R \times S$

1.12 Theta-join $\bowtie_{\theta}$

1.12.1 Definizione

$$R \bowtie_{\theta} S = \sigma_{\theta}(R \times S)$$

- θ è una qualsiasi condizione su attributi di $R \cup S$
- Quando la condizione θ è una congiunzione di atomi di uguaglianza, il theta-join viene chiamato equi-join
- Il self-join si ha quando facciamo la join tra una relazione con se stessa

1.12.2 Esempio

Impiegato	Progetto
Rossi	A
Neri	A
Neri	B

IMPIEGATI

Codice	Nome
A	Venere
B	Marte

PROGETTI

Impiegato	Progetto	Codice	Nome
Rossi	A	A	Venere
Neri	A	A	Venere
Neri	B	B	Marte

$$\text{IMPIEGATI} \bowtie_{\text{Progetto=Codice}} \text{PROGETTI} = \sigma_{\text{Progetto=Codice}}(\text{IMPIEGATI} \times \text{PROGETTI})$$

1.13 Natural-join $\bowtie$

1.13.1 Definizione

$$R(A, B, C) \text{ e } S(B, C, D) \Rightarrow R \bowtie S = \pi_{A,B,C,D} (R \bowtie_{B=B' \wedge C=C'} (\rho_{B',C' \leftarrow B,C}(S)))$$

- Join su tutti gli attributi con lo stesso nome
- Elimina automaticamente le colonne duplicate

1.13.2 Esempio

Impiegato	Reparto
Rossi	Vendite
Neri	Produzione
Verdi	Produzione

R

Reparto	Capo
Produzione	Mori
Vendite	Bruni

S

Impiegato	Reparto	Capo
Rossi	Vendite	Bruni
Neri	Produzione	Mori
Verdi	Produzione	Mori

$R \bowtie S$

1.14 Alberi di esecuzione

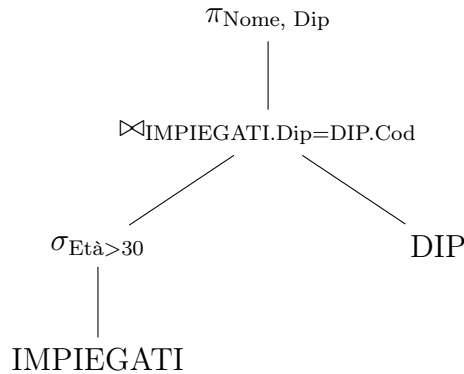
1.14.1 Definizione

Un'espressione si rappresenta come un albero

- Le **foglie** sono relazioni di base
- I **nodi interni** sono operatori di base

1.14.2 Esempio

L'espressione $\pi_{\text{Nome}, \text{Dip}}(\sigma_{\text{Età} > 30}(\text{IMPIEGATI}) \bowtie_{\text{IMPIEGATI.Dip} = \text{DIP.Cod}} \text{DIP})$ ha il seguente albero di esecuzione:



1.15 Divisione $\div$

1.15.1 Definizione

$$R \div S = \{t[X] \mid t \in R \text{ e } \forall s \in S (t[X], s[Y]) \in R\}$$

- **Schema ricorrente:** $R(X, Y)$ e $S(Y) \implies$ risultato sui soli attributi di X
- La divisione si può derivare da altri operatori.
In effetti, siano $R(X, Y)$ e $S(Y)$ due relazioni allora:

$$R \div S = \pi_X((\pi_X(R) \times S) - R)$$

È un operatore non primitivo, ma utile in scrittura

1.15.2 Esempio

$$\pi_{\text{CodImp}}(\text{IMPIEGATO} \div \text{PROG_OBBLIGATORIO})$$

Risultato: Gli impiegati che compaiono con ogni CodProg in PROG_OBBLIGATORIO

1.16 Aggregazione (Y)

1.16.1 Definizione

$$Y_{G;f1(A1) \rightarrow F1, \dots}(R)$$

- G sono gli attributi di raggruppamento
- Le funzioni f sono funzioni di aggregazione (COUNT, SUM, MAX, MIN, AVG)
- $f1(A1) \rightarrow F1$ vuol dire la funzione f1 applica all'attributo A1 e rinomina il risultato come F1

1.16.2 Esempio

- **Schema:**

IMPIEGATO(CodImp, Dip, Stip)

- **Richiesta:** trovare gli stipendi medi per ogni dipartimento
- **Espressione:** $\gamma_{Dip;AVG(Stip) \rightarrow Stip_medio}(IMPIEGATO)$
- **Risultato:** una tupla per Dip con colonna Stip_medio

CodImp	Dip	Stip
1	1	10000
2	1	20000
3	2	5000
4	2	3000
5	2	7000

IMPIEGATO

Dip	Stip_medio
1	15000
2	5000

$\gamma_{Dip;AVG(Stip) \rightarrow Stip_medio}(IMPIEGATO)$

2. DDL - Data Definition Language

2.1 Notazione

- **TERMINI DEL LINGUAGGIO** in maiuscolo
- `<x>` usato per isolare un termine x
- `[x]` indicano che il termine x è opzionale
- `{x}` indicano che il termine x può essere ripetuto da 0 a un numero arbitrario di volte
- `|` separa opzioni alternative

2.2 Schemi

2.2.1 Cosa sono?

Uno schema di base di dati è una collezione di oggetti: domini, tabelle, asserzioni, viste, privilegi, ...

2.2.2 Sintassi di creazione

```
CREATE SCHEMA
  [NomeSchema] -- Se omissso e' il nome dell'utente che ha lanciato il
               comando
  [[authorization] Autorizzazione] -- Termine del linguaggio
  {DefinizioneElementoSchema} -- Termine variabile
```

Dopo il comando **CREATE SCHEMA** compaiono le definizioni dei vari elementi.

Non è necessario che la definizione di tutti gli elementi avvenga contemporaneamente alla creazione dello schema, può avvenire in più fasi successive

2.2.3 Esempio: Creazione di uno schema vuoto

```
CREATE SCHEMA GestioneAzienda;
```

2.2.4 Esempio: Creazione di uno schema utilizzando tutte le opzioni disponibili

```
CREATE SCHEMA GestioneAzienda -- Nome dello schema
AUTHORIZATION utente_admin -- Autorizzazione
CREATE TABLE Impiegato ( -- Termine variabile
  id_impiegato INT PRIMARY KEY,
  nome VARCHAR(50),
```

```

    cognome VARCHAR(50)
)
CREATE VIEW VistaImpiegati AS -- Altro termine variabile
SELECT nome, cognome FROM Impiegato;

```

2.3 Tabelle

2.3.1 Cosa sono?

Una tabella è costituita da una collezione ordinata di attributi e da un insieme (eventualmente vuoto) di vincoli

2.3.2 Sintassi di creazione

```

CREATE TABLE NomeTabella(
    NomeAttributo DOMINIO [ValoreDiDefault][Vincoli]
    {NomeAttributo DOMINIO [ValoreDiDefault][Vincoli]}
    [AltriVincoli]
)

```

2.3.3 Esempio: Creazione di una tabella con elementi minimi

```

CREATE TABLE Studente (
    matricola INT PRIMARY KEY,
    nome VARCHAR(30)
);

```

2.3.4 Esempio: Creazione di una tabella full optional

```

CREATE TABLE Esame (
    codice_esame INT,
    nome_corso VARCHAR(50) NOT NULL,
    data_esame DATE DEFAULT CURRENT_DATE,
    voto INT CHECK (voto BETWEEN 18 AND 30),
    matricola_studente INT,

    -- Definizione della chiave primaria composta
    CONSTRAINT PK_Esame PRIMARY KEY (codice_esame, matricola_studente),

    -- Definizione della chiave esterna con vincolo di integrità
    referenziale
    CONSTRAINT FK_Studente FOREIGN KEY (matricola_studente)
        REFERENCES Studente(matricola)
        ON DELETE CASCADE
        ON UPDATE CASCADE
);

```

2.4 Definizione dei Dati: I Domini

2.4.1 Cosa sono?

I domini specificano i valori ammessi di ciascun attributo

2.4.2 Domini elementari

- **Testuali**

- **varchar(n)**: Stringa di lunghezza variabile tra 0 e n caratteri
- **char(n)**: Stringa di lunghezza pari ad n

- **Numerici**

- **Numerici Esatti**

- * **integer** / **smallint** : rappresentano valori interi
- * **numeric** / **decimal** : rappresentano i valori decimali

- **Numerici approssimati**

- * **real** / **double precision** : rappresentano valori a singola / doppia precisione in virgola mobile
- * **float** permette di richiedere la precisione che si desidera

- **boolean**: un booleano

- **blob, clob**: oggetti di grandi dimensioni

- costruiti da binari (blob)
- costruiti da caratteri (clob)

vengono trattati e memorizzati diversamente dagli altri, vengono usati per informazioni non strutturate o multimediali

2.5 Domini definiti dagli utenti

2.5.1 Cosa sono?

Sono dei domini definiti dall'utente partendo dai domini elementari

2.5.2 Sintassi di creazione

```
CREATE DOMAIN NomeDominio AS DominioElementare  
    [ValoreDefault]  
    [Constraints]
```

2.5.3 Esempio: Creazione di dominio semplice

```
CREATE DOMAIN Voto AS INT;
```

2.5.4 Esempio: Creazione di un dominio full optional

```
CREATE DOMAIN Voto AS SMALLINT
DEFAULT '0'
CHECK (value >= 18 AND value <= 30);
```

2.6 Vincoli intrarelazionale

Un vincolo intrarelazione è una regola che specifica delle condizioni all'interno di una relazione

2.6.1 Tipidi di vincoli intrarelazionali

Proprietà sempre valida all'interno di una relazione

- **NOT NULL:** Il valore deve essere non nullo
- **UNIQUE:** I valori devono essere non ripetuti
- **PRIMARY KEY:** Chiave primaria (una sola per tabella)
- **CHECK:** Definisce condizioni complesse (sia intra che inter relazionali)

2.6.2 Esempio

```
CREATE TABLE Dipendenti (
  -- 1. VINCOLO DI CHIAVE PRIMARIA (e di dominio NOT NULL implicito)
  ID_Dipendente INT PRIMARY KEY,

  -- 2. VINCOLI DI DOMINIO (Tipo dato, NOT NULL e lunghezza)
  Nome VARCHAR(50) NOT NULL,
  Cognome VARCHAR(50) NOT NULL,

  -- 3. VINCOLO DI CHIAVE UNICA (Unique)
  -- L'email deve essere unica per ogni dipendente, ma potrebbe essere
  -- temporaneamente assente (NULL)
  Email VARCHAR(100) UNIQUE,

  -- 4. VINCOLO DI DOMINIO con controllo sul range (CHECK)
  -- Lo stipendio deve essere un numero positivo
  Stipendio_Mensile DECIMAL(10, 2) CHECK (Stipendio_Mensile > 0),

  Data_Assunzione DATE NOT NULL,
  Data_Fine_Contratto DATE,

  -- 5. VINCOLO DI TUPLA (CHECK che coinvolge piu colonne)
  -- La data di fine contratto deve essere successiva alla data di
  -- assunzione
  CONSTRAINT CHK_Date_Contratto CHECK (Data_Fine_Contratto >
    Data_Assunzione)
);
```

2.7 Vincoli interrelazionali: Integrità referenziale

2.7.1 Cosa sono?

Esprimono un legame gerarchico (padre / figlio) fra tabelle.

Alcuni attributi della tabella figlio sono definiti **FOREIGN KEY** si devono riferire (**REFERENCES**) ad alcuni attributi della tabella padre che costituiscono una chiave.

I valori contenuti nella **FOREIGN KEY** devono essere sempre presenti nella tabella padre.

2.7.2 Sintassi

REFERENCES

Nella parte di definizione degli attributi con il costrutto sintattico **REFERENCES**

```
AttrFiglio CHAR(3) REFERENCES TabellaPadre(AttrPadre)
```

FOREIGN KEY

Oppure dopo la definizione degli attributi con i costruttori **FOREGIN KEY** e **REFERENCES**

```
FOREIGN KEY(AttrFiglio)
REFERENCES TabellaPadre(AttrPadre)
```

Quando si hanno più attributi da riferire, si utilizza sempre **FOREIGN KEY**

```
FOREIGN KEY(AttrFiglio1[, AttrFiglio2])
REFERENCES TabellaPadre(AttrPadre1[, AttrPadre2])
```

Se si omettono gli attributi destinazione, vengono assunti quelli della chiave primaria

```
AttrFiglio CHAR(3) REFERENCES TabellaPadre
```

2.7.3 Esempio

```
CREATE TABLE Infrazioni (
    Codice INTEGER PRIMARY KEY, -- Identificativo dell'infrazione (es.
    34321)
    Data DATE NOT NULL,
    Vigile INTEGER NOT NULL,
    Prov CHAR(2),
    Numero CHAR(6),

    -- Vincolo interrelazionale singolo (in linea o a livello tabella)
    FOREIGN KEY (Vigile) REFERENCES Vigili(Matricola),

    -- Vincolo interrelazionale composto (obbligatoriamente a fine
    tabella)
    FOREIGN KEY (Prov, Numero) REFERENCES Auto(Prov, Numero)
);
```

2.8 Aggiornamento

2.8.1 Il problema dell'aggiornamento

Studente				Esame			
Matr	Nome	Città	CDip	Matr	CodCorso	Data	Voto
34321	Luca	Mi	Inf	34321	1	25/02/01	28
53524	Giovanni	To	Mat	34321	2	14/12/01	30
64521	Emilio	Ge	Ing	64521	1	12/03/02	24
73321	Francesca	Vr	Mat	73321	4	18/01/02	18

Se elimino da Studente la tupla con matricola 34321, cosa succede alla tabella esame?

2.8.2 Il problema delle violazioni

Se viene eseguita una operazione di aggiornamento che porta alla violazione di un vincolo di integrità referenziale la base di dati diventa non valida.

Per evitare ciò, vengono definite un insieme di politiche per evitare che questo accada, permettendo di scegliere delle reazioni da adottare in caso di violazione.

Queste politiche vanno dichiarate nella dichiarazione DDL dello schema.

Per tutti gli altri tipi di vincoli invece a seguito di una violazione il comando di aggiornamento viene rifiutato segnalando l'errore.

Tipo di violazione

Inserimento o modifica di tupla che viola il vincolo sulla **tabella figlio** (interna)



Politica di reazione

In entrambi i casi il comando non viene eseguito

Inserimento o modifica di tupla che viola il vincolo sulla tabella padre (esterna)



Quattro politiche possibili

2.8.3 Reazioni a violazioni sulla tabella padre

- **CASCADE:** L'operazione di modifica / cancellazione viene propagata alla tabella intera
- **SET NULL:** NULL nella tabella figlio in entrambi i casi
- **SET DEFAULT:** Default nella tabella figlio in entrambi i casi
- **NO ACTION:** Rifiutata in entrambi i casi

2.8.4 Vincoli con nomi

A fini diagnostici è spesso utile sapere quale vincolo è stato violato a seguito di un'azione sul DB. A tale scopo è possibile associare dei nomi ai vincoli, ad esempio:

```
Stipendio INTEGER CONSTRAINT StipendioPositivo  
CHECK (Stipendio > 0)
```

```
CONSTRAINT ForeignKeySedi  
FOREIGN KEY (Sede) REFERENCES Sedi
```

2.9 Modifiche degli schemi: ALTER

2.9.1 A cosa serve?

Modifica oggetti

2.9.2 Tipi di ALTER

- **ALTER DOMAIN:** Modifica domini
- **ALTER TABLE:** Modifica tabelle

Quando si definisce un nuovo vincolo deve essere soddisfatto dalla istanza presente, altrimenti viene rifiutato

2.9.3 Sintassi ALTER DOMAIN

```
ALTER DOMAIN NomeDominio <  
    SET          DEFAULT ValoreDefault |  
    DROP        DEFAULT |  
    ADD         CONSTRAINT DefVincolo |  
    DROP        CONSTRAINT NomeVincolo >
```

2.9.4 Esempio di ALTER DOMAIN

```
-- 1. Definiamo un valore di default standard per tutti i nuovi  
    inserimenti  
ALTER DOMAIN VotoEsame  
    SET DEFAULT 18;  
-- 2. Aggiungiamo un vincolo per gestire anche l'estensione del voto con  
    la "Lode" (es. inserendo 31)  
ALTER DOMAIN VotoEsame  
    ADD CONSTRAINT ControlloloLode CHECK (VALUE >= 18 AND VALUE <= 31);  
-- 3. Rimuoviamo il valore di default inserito in precedenza se non e piu  
    necessario  
ALTER DOMAIN VotoEsame  
    DROP DEFAULT;
```

```
-- 4. Rimuoviamo il vincolo sulla lode per tornare alle impostazioni base
ALTER DOMAIN VotoEsame
    DROP CONSTRAINT ControlloLode;
```

2.9.5 Sintassi ALTER TABLE

```
ALTER TABLE NomeTabella <
    ALTER COLUMN NomeAttributo <
        SET DEFAULT NuovoDefault |
        DROP DEFAULT > |
    DROP COLUMN NomeAttributo |
    ADD COLUMN DefAttributo |
    DROP CONSTRAINT NomeVincolo |
    ADD CONSTRAINT DefVincolo >
```

2.9.6 Esempio di ALTER TABLE

```
-- 1. Aggiungiamo una nuova colonna per memorizzare l'Email dello
    studente
ALTER TABLE Studente
    ADD COLUMN Email VARCHAR(100);
-- 2. Modifichiamo la colonna "Citta" esistente per impostare un valore
    di default predefinito
ALTER TABLE Studente
    ALTER COLUMN Citta SET DEFAULT 'Mi';
-- 3. Rimuoviamo la colonna "CDip" (Codice Dipartimento) se la tabella
    viene ristrutturata
ALTER TABLE Studente
    DROP COLUMN CDip;
-- 4. Aggiungiamo un vincolo di unicita (Unique) sulla colonna Email
    appena creata
ALTER TABLE Studente
    ADD CONSTRAINT EmailUnica UNIQUE (Email);
-- 5. Rimuoviamo un vecchio vincolo di controllo (se esistente)
    identificato dal suo nome
ALTER TABLE Studente
    DROP CONSTRAINT VerificaMatricola;
```

2.10 Vincolo interrelazionale: UPDATE

Quando si effettua una modifica (UPDATE) sui dati all'interno di una base di dati relazionale, è fondamentale garantire che i legami tra le tabelle rimangano consistenti. Il vincolo di integrità referenziale (Foreign Key) interviene in modo specifico per regolamentare cosa succede quando viene modificata una chiave.

2.10.1 A cosa serve

Il vincolo interrelazionale durante un'operazione di modifica ha lo scopo di impedire la creazione di "tuple orfane". Si considerano due scenari distinti:

- **Modifica sulla tabella figlio (interna):** Se si modifica il valore della chiave esterna nella tabella figlio, il database controlla semplicemente che il nuovo valore esista nella chiave primaria della tabella padre. Se non esiste, l'operazione viene bloccata.
- **Modifica sulla tabella padre (esterna):** Se si modifica il valore della chiave primaria nella tabella padre, tutti i record della tabella figlio che puntavano a quel vecchio valore rimarrebbero associati a una chiave non più esistente. Per evitare questa violazione, il DBMS applica delle specifiche *politiche di reazione*.

Sintassi dell'operazione

La definizione della politica di reazione per la modifica viene dichiarata in fase di creazione o alterazione della tabella figlio, subito dopo la specifica della chiave esterna (**FOREIGN KEY**).

La sintassi generica all'interno del linguaggio SQL DDL è la seguente:

```
FOREIGN KEY (AttributoTabellaFiglio) REFERENCES TabellaPadre(
    AttributoTabellaPadre) ON UPDATE [ RESTRICT | CASCADE | SET NULL | SET
    DEFAULT ]
```

Il significato delle quattro politiche possibili è il seguente:

1. **RESTRICT (o NO ACTION):** È il comportamento di default. La modifica sulla tabella padre viene rifiutata e solleva un errore se esistono tuple collegate nella tabella figlio.
2. **CASCADE:** La modifica viene propagata a cascata. Modificando il valore nella tabella padre, tutti i record corrispondenti nella tabella figlio verranno aggiornati automaticamente con il nuovo valore.
3. **SET NULL:** Il valore della chiave esterna nei record correlati della tabella figlio viene impostato a NULL (richiede che la colonna non abbia il vincolo NOT NULL).
4. **SET DEFAULT:** Il valore della chiave esterna nella tabella figlio viene reimpostato al valore di default precedentemente configurato per quella colonna.

2.10.2 Esempi pratici

Riprendiamo come riferimento le tabelle **Studente** (tabella padre, con chiave primaria **Matr**) ed **Esame** (tabella figlio, con chiave esterna **Matr** che fa riferimento a **Studente**).

Esempio 1: Politica CASCADE (Propagazione automatica)

Si supponga di voler modificare la matricola dello studente 'Luca' da 34321 a 99999. Configurando la tabella con la politica **CASCADE**:

```
ALTER TABLE Esame
    ADD CONSTRAINT fk_studente
    FOREIGN KEY (Matr) REFERENCES Studente(Matrx)
    ON UPDATE CASCADE;
```

Eseguendo l'istruzione `UPDATE Studente SET Matr = 99999 WHERE Matr = 34321;`, il database aggiornerà automaticamente anche tutte le righe all'interno della tabella **Esame** in cui la matricola era 34321, sostituendola con 99999.

Esempio 2: Politica RESTRICT (Blocco cautelativo)

Nel caso in cui le matricole debbano rimanere tassativamente immutabili per ragioni storiche e di sicurezza, si opta per RESTRICT:

```
ALTER TABLE Esame
  ADD CONSTRAINT fk_studente
  FOREIGN KEY (Matr) REFERENCES Studente (Matr)
  ON UPDATE RESTRICT;
```

Se si provasse ad eseguire la modifica della matricola 34321 nella tabella **Studente**, il DBMS bloccherebbe l'operazione restituendo un errore del tipo: *“Violazione del vincolo di chiave esterna: record correlati presenti nella tabella Esame”*.

Esempio 3: Politica SET NULL (Annullamento del legame)

Se l'esame sostenuto deve rimanere registrato nel sistema ma, in seguito alla modifica o riorganizzazione di una targa/codice identificativo padre, si decidesse temporaneamente di scollegarlo dallo studente:

```
ALTER TABLE Esame
  ADD CONSTRAINT fk_studente
  FOREIGN KEY (Matr) REFERENCES Studente (Matr)
  ON UPDATE SET NULL;
```

Cambiando la matricola nella tabella padre, le righe corrispondenti nella tabella **Esame** vedranno il proprio campo **Matr** trasformarsi in NULL, indicando che l'esame è al momento “orfano” di uno studente identificato.

2.11 Vincolo interrelazionale: DROP

2.11.1 Cosa fa?

Cancella oggetti dallo schema

Ha 2 opzioni:

- **RESTRICT:** Un oggetto non è rimosso se non è vuoto
- **CASCADE:** Viene rimosso l'oggetto e tutto ciò che è coinvolto nella definizione dell'oggetto

2.11.2 DROP DOMAIN

Cancellazione del dominio

Sintassi

```
DROP DOMAIN NomeDominio [ RESTRICT | CASCADE ];
```

Esempio di utilizzo

```
-- 1. Creazione del dominio
CREATE DOMAIN CodiceDipartimento AS CHAR(3)
    CHECK (VALUE IS NOT NULL);

-- 2. Utilizzo del dominio nella tabella "Studente"
CREATE TABLE Studente (
    Matr INT PRIMARY KEY,
    Nome VARCHAR(50),
    CDip CodiceDipartimento -- La colonna CDip usa il dominio appena
        creato
);

DROP DOMAIN CodiceDipartimento RESTRICT;
```

DROP TABLE

Cancellazione della tabella

Sintassi

```
DROP TABLE NomeTabella [ RESTRICT | CASCADE ];
```

Esempi

```
-- Creazione della tabella Padre
CREATE TABLE Studente (
    Matr INT PRIMARY KEY,
    Nome VARCHAR(50)
);

-- Creazione della tabella Figlio con il vincolo di Foreign Key
CREATE TABLE Esame (
    CodCorso INT,
    Matr INT,
    Voto INT,
    PRIMARY KEY (CodCorso, Matr),
    CONSTRAINT fk_studente FOREIGN KEY (Matr) REFERENCES Studente(Matrx)
);

DROP TABLE Studente RESTRICT;
-- Nota: Scrivere "DROP TABLE Studente;" produce lo stesso effetto
```

3. Operatore SELECT - Interrogazione di un database

3.1 A cosa serve?

È la istruzione utilizzata per effettuare interrogazioni al database

3.2 Sintassi

```
SELECT ListaAttributi  
FROM ListaTabelle  
[WHERE Condizione]
```

3.3 Esempio

```
SELECT Nome, Cognome, Stipendio  
FROM Impiegato  
WHERE Mansione = 'Ingegnere'
```

3.4 Istruzione AS

3.4.1 A cosa serve?

Viene utilizzato per fare ridenominazioni su attributi e anche su relazioni

3.4.2 Sintassi

```
SELECT AttrEspr [ [ AS ] Alias ] {, AttrEspr [ [ AS ] Alias ] }  
FROM Tabella [ [ AS ] Alias ] {, Tabella [ [ AS ] Alias ] }  
[ WHERE Condizione ]
```

3.4.3 Esempio

```
SELECT I.Nome, D.Citta AS citta_lavoro  
FROM Impiegato AS I, Dip AS D  
WHERE I.Dipart = D.Nome;
```

3.5 Clausola FROM

Specifica la/le tabelle/e su cui operare per determinare il risultato

3.5.1 Sintassi

```
SELECT ListaAttributi  
FROM Tabella1 [ [ AS ] Alias1 ] {, TabellaN [ [ AS ] AliasN ] }
```

3.5.2 Notazione punto

La clausola . permette di specificare a che relazione appartiene un attributo (o anche una tabella a un database)

3.5.3 Esempio

```
SELECT I.Nome , D.NomeDip  
FROM Impiegato AS I, Dip AS D  
WHERE I.Dipart = D.CodDip;
```

3.6 Clausola WHERE

3.6.1 A cosa serve?

È una espressione booleana che specifica una o più condizioni di selezione

3.6.2 Operatori di confronto

- =
- <>
- <= =>
- LIKE
- BETWEEN
- IS NULL
- IS NOT NULL

3.6.3 Operatore LIKE

L'operatore LIKE permette di esprimere dei "pattern" su stringhe di caratteri (regex) mediante la seguente notazione:

- _ indica un singolo carattere arbitrario
- % indica una stringa di caratteri arbitraria (anche lunga 0)

3.6.4 Operatore BETWEEN

L'operatore BETWEEN permette di esprimere condizioni di appartenenza a un intervallo

3.6.5 Operatore IN

L'operatore IN permette di esprimere condizioni di appartenenza a un insieme

3.6.6 Predicati semplici

Sono espressioni di valutazione su attributi mediante operatori di confronto

3.6.7 Precedenza di valutazione tra operatori booleani

1. NOT
2. AND
3. OR

3.6.8 Sintassi

```
WHERE [NOT] PredicatoSemplice {< AND | OR > [NOT] PredicatoSemplice}
```

3.6.9 Esempio generico

```
SELECT Nome, Cognome, Mansione
FROM Impiegato
WHERE ufficio = '10';
```

3.6.10 Esempio con LIKE

```
SELECT Nome, Cognome
FROM Impiegato
WHERE Cognome LIKE '%i';
```

Figura 3.1: Restituire tutti i cognomi che finiscono con la lettera i

```
SELECT Nome, Cognome
FROM Impiegato
WHERE Cognome LIKE 'R_ssi';
```

Figura 3.2: Restituirà "Rossi" (dove _ sostituisce la 'o'), ma non "Rossini" (perché _ impone un solo carattere)

3.6.11 Esempio con BETWEEN

```
SELECT Nome, Stipendio
FROM Impiegato
WHERE Stipendio BETWEEN 38 AND 60;
```

3.6.12 Esempio con IN

```
SELECT Cognome, ufficio
FROM Impiegato
WHERE ufficio IN ('10', '20');
```

3.7 Clausola DISTINCT

3.7.1 A cosa serve?

In SQL, le tabelle prodotte dalle interrogazioni possono contenere più righe identiche tra loro. I duplicati possono essere rimossi usando la parola chiave DISTINCT

3.7.2 Sintassi

```
SELECT DISTINCT ListaAttributi
FROM ListaTabelle
[ WHERE Condizione ]
```

3.7.3 Esempio

```
SELECT DISTINCT Cognome, Citta
FROM Impiegato
WHERE Stipendio > 40;
```

3.8 Espressioni

3.8.1 A cosa servono?

Una espressione usata nella clausola SELECT dà luogo ad una nuova "colonna" che non corrisponde a nessuna colonna della tabella su cui si effettua la selezione.

Questa nuova colonna è detta colonna virtuale.

Le colonne virtuali non sono fisicamente memorizzate, ma esistono solo come risultato delle interrogazioni.

Anche alle colonne virtuali è possibile assegnare un alias (con il costrutto AS)

3.8.2 Funzioni predefinite

SQL mette a disposizione alcune funzioni predefinite:

- **Stringhe:** UPPER, UCASE, LENGTH, ...
- **Date / Intervalli:** +, -, DATE, DAYOFWEEK, ...
- **Matematiche:** *, +, -, /, TAN, SQRT, SIN, ...
- **Informazioni di sistema:** USER, CURRENT_DATE, ...

3.8.3 Funzioni per stringhe

Gli operatori più comuni per le stringhe sono:

- **||:** Operatore di concatenazione
- **UPPER, UCASE:** Trasforma la stringa in caratteri maiuscoli
- **LOWER, LCASE:** Trasforma la stringa in caratteri minuscoli

3.8.4 Esempio di utilizzo di funzioni aritmetiche

```
SELECT Nome, Stipendio, Premioprod, (Stipendio + Premioprod) AS  
       stipendio_totale  
FROM Impiegato  
WHERE Mansione = 'Ingegnere';
```

3.8.5 Esempio di utilizzo di funzioni per stringhe

```
SELECT Nome, Cognome, citta, mansione  
FROM Impiegato  
WHERE UCASE(mansione) = 'INGEGNERE';
```


4. Operatore JOIN

4.1 A cosa serve?

L'operatore JOIN rappresenta un'importante funzionalità in quanto permette di correlare dati in tabelle diverse.

Matematicamente si tratta del prodotto cartesiano tra le 2 tabelle, che avrà un numero di righe pari al prodotto del numero di righe delle due tabelle.

Ad un operatore JOIN sono applicati uno o più predicati join.

4.1.1 Cosa è un predicato join?

Un predicato di join esprime una condizione che deve essere verificata dalle tuple del risultato dell'interrogazione.

Il predicato di join permette di estrarre dal DB un sottoinsieme opportuno del prodotto cartesiano.

4.2 Sintassi JOIN Legacy

Nella sintassi legacy, le tabelle da unire vengono elencate nella clausola FROM separate da una virgola, e la condizione di join viene specificata nella clausola WHERE

```
SELECT ListaAttributi
FROM Tabella1, Tabella2
WHERE Tabella1.Attributo = Tabella2.Attributo;
```

4.2.1 Esempio

```
SELECT Impiegato.Nome, Dip.Citta AS citta_lavoro
FROM Impiegato, Dip
WHERE Impiegato.Dipart = Dip.Nome;
```

4.3 Sintassi JOIN SQL-2

Lo standard SQL-2 ha introdotto una sintassi dedicata che separa chiaramente la logica di unione (la JOIN) dalle condizioni di filtro (la WHERE), rendendo le interrogazioni più leggibili e riducendo il rischio di errori nel dimenticare le clausole di collegamento

```
SELECT ListaAttributi
FROM Tabella1 [AS Alias1]
JOIN Tabella2 [AS Alias2] ON Tabella1.Attributo = Tabella2.Attributo;
```

4.3.1 Esempio

```
SELECT I.Nome, D.Citta AS citta_lavoro
FROM Impiegato AS I
JOIN Dip AS D ON I.Dipart = D.Nome;
```

4.4 OUTER JOIN

4.4.1 Cosa fa?

L'operatore OUTER JOIN è un'estensione della JOIN standard (Inner Join) utilizzata quando si desidera conservare nel risultato anche le tuple che non hanno corrispondenze nell'altra tabella.

Mentre l'Inner Join restituisce solo le righe che soddisfano la condizione di unione (ovvero quelle con corrispondenza in entrambe le tabelle), l'Outer Join garantisce che almeno una delle due tabelle (o entrambe) sia rappresentata completamente.

Esistono 3 tipologie di OUTER JOIN

4.4.2 LEFT OUTER JOIN (o LEFT JOIN)

Definizione

Restituisce tutte le righe della tabella indicata a sinistra, per le righe della tabella che non hanno corrispondenza nella tabella di destra, vengono visualizzati valori NULL per tutti gli attributi provenienti dalla tabella di destra

Sintassi

```
SELECT colonne
FROM TabellaA A
LEFT JOIN TabellaB B ON A.chiave = B.chiave;
```

Esempio

```
SELECT I.Nome, D.Citta
FROM Impiegato I
LEFT JOIN Dip D ON I.Dipart = D.Nome;
```

4.4.3 RIGHT JOIN

Definizione

Restituisce tutte le righe della tabella indicata a destra, per le righe della tabella che non hanno corrispondenza nella tabella di sinistra, vengono visualizzati valori NULL per tutti gli attributi provenienti dalla tabella di sinistra

Sintassi

```
SELECT I.Nome, D.NomeDipartimento
FROM Impiegato I
RIGHT JOIN Dip D ON I.Dipart = D.ID_Dipartimento;
```

Esempio

```
SELECT I.Nome, D.NomeDipartimento
FROM Impiegato I
RIGHT JOIN Dip D ON I.Dipart = D.NomeDipartimento;
```

FULL JOIN

Definizione

La FULL JOIN restituisce tutte le righe provenienti da entrambe le tabelle coinvolte nell'operazione. Quando non esiste una corrispondenza tra una riga di una tabella e l'altra, il risultato includerà comunque la riga, popolando con valori NULL tutti gli attributi provenienti dalla tabella in cui manca il riscontro.

Sintassi

```
SELECT colonne
FROM TabellaA A
FULL JOIN TabellaB B ON A.chiave = B.chiave;
```

Esempio

```
SELECT I.Impiegato, D.Dip
FROM Impiegato I
FULL JOIN Dip D ON I.Dipart = D.Dip;
```

4.5 Self Join

4.5.1 Cosa è?

La Self Join (o auto-congiunzione) è un caso speciale di JOIN in cui una tabella viene congiunta con se stessa.

Non esiste un comando SQL chiamato SELF JOIN: si utilizza la sintassi classica della INNER JOIN o della LEFT JOIN, ma riferendosi alla stessa tabella sia a destra che a sinistra dell'operatore.

4.5.2 A cosa serve?

Serve quando i dati memorizzati in una riga hanno una relazione con i dati memorizzati in un'altra riga della stessa tabella.

4.5.3 Sintassi

```
SELECT <colonne>
FROM <nome_tabella> AS <alias1>
[INNER | LEFT] JOIN <nome_tabella> AS <alias2>
    ON <alias1>.<colonna_connessione> = <alias2>.<colonna_chiave>;
```

4.5.4 Esempio

```
SELECT
    E.Nome AS Impiegato,
    M.Nome AS Manager
FROM Impiegato AS E
LEFT JOIN Impiegato AS M
    ON E.ID_Manager = M.ID;
```

4.6 Join Esplicito

4.6.1 Cosa è?

Il join esplicito è un approccio che definisce l'unione tra tabelle usando un'apposita sintassi dedicata.

4.6.2 A cosa serve?

A livello logico, serve a separare nettamente la struttura di collegamento tra le tabelle dai filtri di selezione sui dati.

La clausola specifica direttamente quali tabelle unire e isola la condizione di join. Questo riduce drasticamente il rischio di errori distruttivi e aumenta la leggibilità del codice. Inoltre, garantisce la flessibilità necessaria a implementare facilmente vari tipi di unione esterna (come LEFT, RIGHT o FULL JOIN).

4.6.3 Sintassi

```
SELECT colonna1, colonna2, ...
FROM TabellaA [AliasA]
JOIN TabellaB [AliasB]
    ON AliasA.chiave_esterna = AliasB.chiave_primaria
[WHERE altre_condizioni_di_filtro];
```

4.6.4 Esempio

```
SELECT I.Nome, D.Citta
FROM Impiegato AS I
JOIN Dip AS D
    ON I.Dipart = D.Nome;
```

5. Ordinamento dei risultati

5.1 ORDER BY

5.1.1 Cosa è?

L'ORDER BY è una clausola opzionale che si inserisce al termina di una query SELECT per ordinare le righe del risultato in base ai valori di una o più colonne

5.1.2 Sintassi

```
SELECT colonna1, colonna2, ...  
FROM NomeTabella  
[WHERE condizioni_di_filtro]  
ORDER BY colonnaA [ASC | DESC], colonnaB [ASC | DESC];
```

Ordinamento

- **Ascendente (ASC)** è default e può essere omesso
- **Discendente (DESC)**

5.1.3 Esempio

```
SELECT  
    UCASE(I.Nome) AS Nome_Impiegato,  
    ((I.Stipendio + I.Premio) * 13) AS Guadagno_Annuo_Stimato,  
    D.Citta AS Sede_Dipartimento  
FROM Impiegato AS I  
JOIN Dip AS D  
    ON I.Dipart = D.Nome  
WHERE UCASE(I.Mansione) = 'INGEGNERE'  
ORDER BY Guadagno_Annuo_Stimato DESC, Nome_Impiegato ASC;
```

5.2 Operatori di aggregazione

5.2.1 Quali sono?

- **COUNT:** conta numero di tuple di tabella
- **SUM:** somma i valori o espressioni di attributi
- **MAX:** valore massimo di un attributo di tabella
- **MIN:** valore minimo di un attributo di tabella

- **AVG:** valore medio di un attributo di tabella
- **GROUP BY:** raggruppamento delle tuple in sottogruppi gestiti come estensione delle normali interrogazioni

5.2.2 Come vengono applicati?

1. Si esegue l'interrogazione sulla base di clausole FROM e WHERE
2. Si applica l'operatore aggregato alla tabella risultato dell'interrogazione

5.3 COUNT

5.3.1 Cosa fa?

Restituisce il numero di valori degli attributi in ListaAttr

5.3.2 Sintassi

```
COUNT (< * | [DISTINCT | ALL] ListaAttr >)
```

- **DISTINCT:** numero di valori distinti diversi da NULL di ListaAttr
- **ALL:** numero di valori diversi da NULL di ListaAttr
- **COUNT (*):** conta numero di righe di una tabella

Se non è specificato nulla, ALL è l'opzione di default

5.3.3 Esempio

```
SELECT
    D.Nome AS Nome_Dipartimento,
    D.Citta AS Sede,
    COUNT(I.Matricola) AS Numero_Impiegati
FROM Dip AS D
JOIN Impiegato AS I
    ON D.Nome = I.Dipart
WHERE I.Stipendio > 1500
GROUP BY D.Nome, D.Citta
HAVING COUNT(I.Matricola) > 2
ORDER BY Numero_Impiegati DESC;
```

5.4 SUM

5.4.1 Cosa fa?

Restituisce la somma dei valori di AttrExpr relativamente alle tuple di tabella specificate nella query, opera su attributi numerici

5.4.2 Sintassi

```
SELECT SUM(nome_colonna) AS nome_alias
FROM NomeTabella
[WHERE condizioni_di_filtro];
```

5.4.3 Esempio

```
SELECT SUM(Stipendio) AS Spesa_Totale_Stipendi
FROM Impiegato;
```

5.5 MIN/MAX

5.5.1 Cosa fa?

Restituisce il massimo/minimo dei valori di AttrExpr relativamente alle tuple di tabella specificate nella query, opera su attributi ordinabili

5.5.2 Sintassi

```
SELECT
    MIN(nome_colonna) AS minimo_alias ,
    MAX(nome_colonna) AS massimo_alias
FROM NomeTabella
[WHERE condizioni_di_filtro];
```

5.5.3 Esempio

```
SELECT
    MIN(Stipendio) AS Stipendio_Minimo ,
    MAX(Stipendio) AS Stipendio_Massimo
FROM Impiegato;
```

5.6 AVG

5.6.1 Cosa fa?

Restituisce la media dei valori di AttrExpr relativamente alle tuple di tabella specificate nella query, opera su valori numerici

5.6.2 Sintassi

```
SELECT AVG(nome_colonna) AS nome_alias
FROM NomeTabella
[WHERE condizioni_di_filtro];
```

5.6.3 Esempio

```
SELECT AVG(Stipendio) AS Stipendio_Medio_Aziendale
FROM Impiegato;
```

5.7 Target List

SQL impedisce di includere in una stessa target list funzioni aggregate e espressioni al livello di riga

```
SELECT Cognome, MAX(Stipendio) -- Questa cosa non si puo fare
FROM Impiegato
WHERE Dipart = "Amministrazione"
```

È necessario ricorrere alle subquery, che si vedranno più avanti

5.8 Operatore di aggregamento GROUP BY

5.8.1 Cosa è?

La GROUP BY è una clausola dell'istruzione SELECT utilizzata per raggruppare le righe di una tabella che condividono gli stessi valori in una o più colonne specificate.

5.8.2 Sintassi

```
SELECT colonna_di_raggruppamento, FUNZIONE_AGGREGATA(altra_colonna)
FROM NomeTabella
[WHERE filtri_sulle_singole_righe]
GROUP BY colonna_di_raggruppamento
[HAVING filtri_sui_gruppi_creati]
[ORDER BY criterio_di_ordinamento];
```

5.8.3 Esempio

```
SELECT Dipart, AVG(Stipendio) AS Stipendio_Medio
FROM Impiegato
GROUP BY Dipart;
```

5.9 Condizioni WHERE e HAVING

Specificano entrambi condizioni di selezione:

- **WHERE:** Predicati che operano su tuple individuabili (seleziona di tuple prima di GROUP BY)
- **HAVING:** Predicati che operano su gruppi di tuple creati da GROUP BY (selezioni raggruppamenti)

5.9.1 Sintassi WHERE

```
WHERE attributo OPERATORE valore_o_costante  
[ AND | OR | NOT altra_condizione ];
```

5.9.2 Esempio di uso di WHERE

```
SELECT Cognome, Stipendio  
FROM Impiegato  
WHERE Dipart = 'Amministrazione' AND Stipendio > 1500;
```

5.9.3 Sintassi HAVING

```
GROUP BY colonna_raggruppata  
HAVING FUNZIONE_AGGREGATA(colonna) OPERATORE valore;
```

5.9.4 Esempio di uso di HAVING

```
SELECT Dipart, AVG(Stipendio) AS AvgStipendio  
FROM Impiegato  
GROUP BY Dipart  
HAVING COUNT(*) >= 2; -- Mantiene solo i dipartimenti con almeno 2  
impiegati
```

6. Interrogazioni di tipi insiemistico

6.1 Operatori insiemistici

Permettono di costruire query concatenando due query SQL

- UNION
- INTERSECT
- EXCEPT (Differenza)

6.1.1 Sintassi

```
SelectSQL{< UNION / INTERSECT / EXCEPT > [ALL] SelectSQL}
```

I duplicati vengono eliminati (a meno che si usi ALL)

6.2 UNION

6.2.1 Cosa fa?

Esegue l'unione dei risultati delle 2 query.

L'operatore UNION impone alcune restrizioni sulle interrogazioni su cui opera:

- Le interrogazioni devono restituire lo stesso numero di colonne, e le colonne corrispondenti devono avere lo stesso dominio o domini compatibili
- La corrispondenza si basa sulla posizione delle colonne indipendentemente dal loro nome
- Se si usa la clausola di ORDER BY queste deve essere usata una sola volta alla fine dell'interrogazione e non alla fine di ogni select

6.2.2 Sintassi

```
SelectSQL{ UNION [ALL] SelectSQL}
```

6.2.3 Esempio

```
SELECT CodOrd  
FROM Ordine  
WHERE Importo > 500  
  
UNION  
  
SELECT CodOrd
```

```
FROM Dettaglio  
WHERE Qta > 1000;
```

6.3 INTERSECT

6.3.1 Cosa fa?

Esegue l'intersezione dei risultati delle 2 query.

L'operatore INTERSECT impone alcune restrizioni sulle interrogazioni su cui opera:

- Le interrogazioni devono restituire lo stesso numero di colonne, e le colonne corrispondenti devono avere lo stesso dominio o domini compatibili
- La corrispondenza si basa sulla posizione delle colonne indipendentemente dal loro nome
- Se si usa la clausola di ORDER BY queste deve essere usata una sola volta alla fine dell'interrogazione e non alla fine di ogni select

6.3.2 Sintassi

```
SelectSQL{ INTERSECT [ALL] SelectSQL }
```

6.3.3 Esempio

```
SELECT CodOrd  
FROM Ordine  
WHERE Importo > 500  
  
INTERSECT  
  
SELECT CodOrd  
FROM Dettaglio  
WHERE Qta > 1000;
```

6.4 EXCEPT

6.4.1 Cosa fa?

Esegue la differenza dei risultati delle 2 query.

L'operatore EXCEPT impone alcune restrizioni sulle interrogazioni su cui opera:

- Le interrogazioni devono restituire lo stesso numero di colonne, e le colonne corrispondenti devono avere lo stesso dominio o domini compatibili
- La corrispondenza si basa sulla posizione delle colonne indipendentemente dal loro nome
- Se si usa la clausola di ORDER BY queste deve essere usata una sola volta alla fine dell'interrogazione e non alla fine di ogni select

6.4.2 Sintassi

```
SelectSQL { EXCEPT [ ALL ] SelectSQL }
```

6.4.3 Esempio

```
SELECT Nome  
FROM Impiegato  
  
EXCEPT  
  
SELECT Cognome AS Nome  
FROM Impiegato;
```

7. Interrogazioni Nidificate

7.1 Cosa sono?

Una delle ragioni che rendono SQL un linguaggio potente è la possibilità di esprimere Interrogazioni più complesse in termini di interrogazioni più semplici, tramite il meccanismo delle subqueries

7.2 Come funzionano?

La clauso WHERE di una query può contenere un'altra query.

La subquery viene usata per determinare uno o più valori da usare come valori di confronto in un predicato della query esterna

7.3 Sintassi Subquery

```
SELECT <lista_attributi>
FROM <tabella>
WHERE <valore_scalare> <operatore_confronto> [ANY | ALL]
    (
        SELECT <colonna>
        FROM <tabella_sottoquery>
        [WHERE <condizioni>]
    );
```

7.4 Predicati ANY e ALL

Nella clausola WHERE possono comparire predicati che confrontano un attributo (o un'espressione sugli attributi) con il risultato di una query SQL

- **ANY:** Il predicato è vero se almeno una riga restituita dalla query SelectSQL soddisfa il controllo
- **ALL:** Il predicato è vero se tutte le righe restituite dalla query SelectSQL soddisfano il confronto

7.4.1 Sintassi

```
ScalarValue Operator <ANY | ALL> SelectSQL
```

L'Operato può essere uno tra : =, <>, <, <=, >, >=

7.4.2 Esempio

ANY

```
SELECT Cognome
FROM Impiegato
WHERE Dipart = ANY (SELECT Nome
                    FROM Dip
                    WHERE Citta = 'Milano');
```

ALL

```
SELECT Nome, Cognome
FROM Impiegato
WHERE Stipendio >= ALL (SELECT Stipendio
                       FROM Impiegato);
```

7.5 Quantificatori esistenziali

EXISTS e NOT EXISTS ammettono come parametro le query nidificate.

7.5.1 EXISTS

L'operatore logico EXISTS restituisce il valore booleano true se la subquery restituisce almeno una tupla, false altrimenti

Sintassi

```
EXISTS SelectStar
```

Esempio

```
SELECT A.Citta, A.Nazione
FROM AEROPORTO AS A
WHERE EXISTS (SELECT * FROM VOLO AS V
              WHERE V.CittaPart = A.Citta);
```

7.5.2 NOT EXISTS

L'operatore NOT EXISTS restituisce il valore booleano true se la subquery non restituisce alcuna tupla, false altrimenti

Sintassi

```
NOT EXISTS SelectStar
```

Esempio

```
SELECT O.CodOrd
FROM Ordine AS O
WHERE NOT EXISTS (
    SELECT * FROM Prodotto AS P
    WHERE NOT EXISTS (
        SELECT * FROM Dettaglio AS D
        WHERE D.CodOrd = O.CodOrd
            AND D.CodProd = P.CodProd
    )
);
```

8. Viste

8.1 Cosa sono?

Una vista è una relazione virtuale attraverso è possibile vedere i dati memorizzati nella relazioni reali, chiamate relazioni di base.

8.2 Proprietà fondamentali

- **Natura virtuale:** Una vista non contiene tuple fisiche al suo interno. I suoi dati vengono materializzati dinamicamente eseguendo l'interrogazione (query) che la definisce
- **Utilizzo:** Può essere utilizzata (quasi) nello stesso modo in cui si usa una normale relazione di base
- **Definizione:** Viene definita mediante un'integrazione su una o più relazioni di base, oppure basandosi su altre viste preesistenti. Si crea associando un nome univoco e una lista di attributi al risultato di tale integrazione

8.3 A cosa servono?

Il meccanismo delle viste è particolarmente utile per due scopi principali:

- **Semplificare l'accesso ai dati:** Permette di salvare query complesse sotto un unico nome, rendendo le successive interrogazioni molto più semplici.
- **Garantire la privacy dei dati:** Consente di mostrare a determinati utenti solo una parte specifica dei dati (un sottoinsieme di righe o colonne), nascondendo il resto della relazione di base

8.4 Sintassi

```
CREATE VIEW NomeVista [(ListaAttributi)] AS query
```

- **NomeVista:** È il nome della vista che viene create, tale nome deve essere unico rispetto a tutti i nomi di relazioni e di viste definite dallo stesso utente che la definisce
- **query:** È l'interrogazione di definizione della vista. Una vista ha lo stesso numero di colonne pari alle colonne (di base o virtuali) specificate nella clausola di proiezione di query
- **ListaAttributi:** È una lista di nomi da assegnare alle colonne della vista, tale specifica non è obbligatoria, tranne nel caso in cui l'interrogazione contenga nella clausola di proiezioni funzioni di gruppo e/o espressioni

8.5 Esempio creazione vista

```
CREATE VIEW Personale AS  
SELECT Nome, Mansione, Pnome  
FROM Impiegati, Progetti  
WHERE Impiegati.ProgNo = Progetti.ProgNo;
```